

**EXERCICE 1.1 :**

```

#include <stdio.h>
int main()
{
    int N,I;
    long SOM;
    do
    {
        printf("Dimension du tableau (max.50) : ");
        scanf("%d", &N );
    }while(N<0 || N>50);

    int T[N];

    /* Lecture du tableau */
    for (I=0; I<N; I++)
    {
        printf("Elément %d : ", I);
        scanf("%d", &T[I]);
    }

    /* Affichage du tableau */
    printf("Tableau donné :\n");
    for (I=0; I<N; I++)
        printf("%d ", T[I]);

    printf("\n");
    for (I=0; I<N; I++)
        SOM+=T[I];

    printf("Somme des elements : %d\n",SOM);
}

```

**EXERCICE 1.2 :**

```

#include <stdio.h>

int main()
{
    /* Déclarations */
    /* Les tableaux et leurs dimensions */
    int T[50], TPOS[50], TNEG[50];
    int N,  NPOS,  NNEG;
    int I; /* indice courant */

```

```

/* Saisie de dimension */
do{
printf("Dimension du tableau N =[0,50]");
scanf("%d", &N );
} while(N<0 || N>50);
/* int T[N], TPOS[N], TNEG[N];*/
/* Lecture du tableau */
for (I=0; I<N; I++)
{
    printf("Elément %d : ", I);
    scanf("%d", &T[I]);
}

/* Affichage du tableau */

printf("Tableau donné :\n");
for (I=0; I<N; I++)
    printf("%d ", T[I]);

printf("\n");

/* Initialisation des dimensions de TPOS et TNEG */
NPOS=0;
NNEG=0;
/* Transfer des données */
for (I=0; I<N; I++)
{
if (T[I]>0) {
        TPOS[NPOS]=T[I];
        NPOS++;
    }
if (T[I]<0) {
        TNEG[NNEG]=T[I];
        NNEG++;
    }
}

/* Edition du résultat */
printf("Tableau TPOS :\n");
for (I=0; I<NPOS; I++)
    printf("%d ", TPOS[I]);

printf("\n");

printf("Tableau TNEG :\n");
for (I=0; I<NNEG; I++)
    printf("%d ", TNEG[I]);

return 0;

```

```
}
```

### **EXERCICE 1.3 :**

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int N,I,j;
```

```
long SOM;
```

```
do
```

```
{
```

```
printf("Dimension du tableau (max.50) : ");
```

```
scanf("%d", &N );
```

```
}while(N<0 || N>50);
```

```
int T[N];
```

```
/* Lecture du tableau */
```

```
for (I=0; I<N; I++)
```

```
{
```

```
printf("Elément %d : ", I);
```

```
scanf("%d", &T[I]);
```

```
}
```

```
/* Affichage du tableau */
```

```
printf("Tableau donné :\n");
```

```
for (I=0; I<N; I++)
```

```
printf("%d ", T[I]);
```

```
printf("\n");
```

```
for (I=0,j=0; I<N; I++)
```

```
{
```

```
    T[j]=T[I];
```

```
    if(T[I]!=0) j++;
```

```
}
```

```
/* solution 2 */
```

```
for (I=0,j=0; I<N; I++)
```

```
{
```

```
    if(T[I]!=0) {
```

```
        T[j]=T[I];
```

```
        j++;
```

```
    }
```

```

    }
    N=j;
for (I=0; I<N; I++)
    printf("%d ", T[I]);
}

```

#### EXERCICE 1.4 :

```

#include <stdio.h>
int main()
{
    /* Déclarations */
    int A[50]; /* tableau donné */
    int N; /* dimension */
    int I; /* indice courant */
    int MIN; /* position du minimum */
    int MAX; /* position du maximum */
    /* Saisie des données */
    printf("Dimension du tableau (max.50) : ");
    scanf("%d", &N );
    for (I=0; I<N; I++)
    {
        printf("Elément %d : ", I);
        scanf("%d", &A[I]);
    }
    /* Affichage du tableau */
    printf("Tableau donné :\n");
    for (I=0; I<N; I++)
        printf("%d ", A[I]);
    printf("\n");
    /* Recherche du maximum et du minimum */
    MIN=0;
    MAX=0;
    for (I=0; I<N; I++)
    {
        if(A[I]>A[MAX]) MAX=I;
        if(A[I]<A[MIN]) MIN=I;
    }
    /* Edition du résultat */
    printf("Position du minimum : %d\n", MIN);
    printf("Position du maximum : %d\n", MAX);
    printf("Valeur du minimum : %d\n", A[MIN]);
    printf("Valeur du maximum : %d\n", A[MAX]);
    return 0;
}

```

#### Exercice 1.5.

```

#include <stdio.h>
int main()

```

```

{
int N,I,j, PMAX, AIDE;
long SOM;
do
{
printf("Dimension du tableau (max.50) : ");
scanf("%d", &N );

}while(N<0 || N>50);

int A[N];

/* Lecture du tableau */
for (I=0; I<N; I++)
{
printf("Élément %d : ", I);
scanf("%d", &A[I]);
}

/* Affichage du tableau */

printf("Tableau donné :\n");
for (I=0; I<N; I++)
printf("%d ", A[I]);

printf("\n");
for (j=0; j<N-1; j++)
{
PMAX=j;
for (I=j+1; I<N; I++)
if(A[I]>A[PMAX]) PMAX=I;
AIDE = A[j];
A[j]=A[PMAX];
A[PMAX]=AIDE;
}

printf("Tableau trié: \n");

for (I=0; I<N; I++)
printf("%d ", A[I]);
}

```

**Exercice 2.1 et 2.2 :**

```
#include <stdio.h>
int main()
{
    /* Déclarations */
    int T[50][50]; /* tableau donné */
    int L, C;      /* dimensions */
    int I, J;      /* indices courants */
    long SOM; /* somme des éléments - type long à cause */
               /* de la grandeur prévisible des résultats. */

    /* Saisie des données */
    do{

        printf("Nombre de lignes (max.50) : ");
        scanf("%d", &L );
        printf("Nombre de colonnes (max.50) : ");
        scanf("%d", &C );}while(L>50||C>50);
    for (I=0; I<L; I++)
        for (J=0; J<C; J++)
        {
            printf("Elément[%d][%d] : ",I,J);
            scanf("%d", &T[I][J]);
        }
    /* Affichage du tableau */
    printf("Tableau donné :\n");
    for (I=0; I<L; I++)
    {
        for (J=0; J<C; J++)
            printf("%7d", T[I][J]);
        printf("\n");
    }
    /* Calcul et affichage de la somme des elements*/
    SOM=0;
    for (I=0; I<L; I++)
    {
        for (J=0; J<C; J++)
            SOM += T[I][J];

    }
    printf("Somme des elements %ld\n",SOM);
    /* Calcul et affichage de la somme des lignes */
    for (I=0; I<L; I++)
    {
```

```

    for (SOM=0, J=0; J<C; J++)
        SOM += T[I][J];
    printf("Somme - ligne %d : %ld\n",I,SOM);
}
/* Calcul et affichage de la somme des colonnes */
for (J=0; J<C; J++)
{
    for (SOM=0, I=0; I<L; I++)
        SOM += T[I][J];
    printf("Somme - colonne %d : %ld\n",J,SOM);
}
return 0;
}

```

### Exercice 2.3

```

#include <stdio.h>
int main()
{
    /* Déclarations */
    int M[10][10]; /* tableau à 2 dimensions */
    int V[100];    /* tableau à 1 dimension */
    int L, C;      /* dimensions */
    int I, J,j;    /* indices courants */

    /* Saisie des données */
    printf("Nombre de lignes (max.10) : ");
    scanf("%d", &L );
    printf("Nombre de colonnes (max.10) : ");
    scanf("%d", &C );

    for (I=0; I<L; I++)
        for (J=0; J<C; J++)
        {
            printf("Elément[%d][%d] : ",I,J);
            scanf("%d", &M[I][J]);
        }

    /* Affichage du tableau 2-dim */
    printf("Tableau donné :\n");
    for (I=0; I<L; I++)
    {
        for (J=0; J<C; J++)
            printf("%d ", M[I][J]);
        printf("\n");
    }
}

```

```
}
```

```
/* Transfer des éléments ligne par ligne 1er solution */
```

```
for (I=0; I<L; I++)
```

```
{
```

```
for (J=0; J<C; J++)
```

```
{
```

```
    V[I*C+J] = M[I][J];
```

```
}
```

```
}
```

```
//deuxième solution
```

```
j=0;
```

```
for (I=0; I<L; I++)
```

```
    for (J=0; J<C; J++)
```

```
    {
```

```
        V[j] = M[I][J];
```

```
        j++;}
```

```
/* Affichage du tableau 1-dim */
```

```
printf("Tableau résultat : ");
```

```
for (I=0; I<L*C; I++)
```

```
{
```

```
    printf("%d ", V[I]);
```

```
}
```

```
printf("\n");
```

```
return 0;
```

```
}
```